

Architectural Audit Report

Codebase, Security, Roles, & Database Verification

Date: October 8, 2026

Target Repo: Mohit616-byte/Havli

Status: Complete (Read-Only)

Executive Summary: Havli is a social house-party and event ticketing web application focused on Delhi NCR (Gurgaon, Delhi, Noida, etc.). The platform is implemented on **Next.js 16.3.0**, **React 19.2.8**, **Tailwind CSS v4**, and **Supabase (PostgreSQL 15+)**. This document delivers the comprehensive structural and architectural audit of the entire codebase.

1 Project Folder Structure

Havli follows an enterprise layered architecture with clear boundaries between UI views, route handlers, business services, and database repositories:

```
c:\Havli\
├── app/                                # Next.js App Router (Pages, Layouts, API Handlers)
│   ├── layout.tsx                     # Root HTML layout with dark theme & AuthProvider
│   ├── page.tsx                       # Landing page (Hero, Locations, Vibes, Featured)
│   ├── globals.css                    # Global Tailwind CSS v4 variables and theme tokens
│   ├── (auth routes)/                 # login/, signup/, onboarding/, profile/, auth/confirm/
│   ├── explore/                       # Explore discovery catalog with multi-facet filters
│   ├── events/[id]/                   # Event details, capacity bar, and Book Your Spot CTA
│   ├── bookings/ & bookings/[id]/     # User activity, ticket passes, and booking receipts
│   ├── host/ & host/dashboard/         # Host submissions, event management, and guest rosters
│   ├── admin/page.tsx                 # Admin portal: pending host submission queue
│   └── api/                           # Route handlers: admin/, auth/, bookings/, events/, host/
├── components/                        # Reusable UI Components
│   ├── ui/                           # Atoms: Button, Input, Select, Textarea, Modal
│   ├── layout/                       # Navbar, MobileNav, Footer
│   ├── providers/                    # AuthProvider (Supabase session & user profile state)
│   ├── home/                         # Hero, LocationSection, VibeSection, FeaturedEvents
│   └── events/ & explore/ & host/     # EventCard, EventGrid, FilterBar, HostForm
├── lib/                              # Domain logic, database access & utilities
│   ├── mock-data.ts                  # Fallback data, vibe tags, and Delhi NCR cities
│   ├── utils/profile.ts               # Profile completion validator (isProfileComplete)
│   ├── supabase/                     # Browser singleton (client.ts) & server auth extractor (server.ts)
│   ├── server/                       # Layered Architecture: repositories/ & services/
│   └── supabase/migrations/           # Ordered SQL schema migrations (0000000 to 0000006)
```

2 Next.js Version and Routing Approach

- **Core Stack:** Next.js 16.3.0 (App Router) paired with React 19.2.8 and Tailwind CSS v4 (via @tailwindcss/postcss).
- **Server / Client Segregation:** Top-level route pages define SEO metadata and Suspense wrappers server-side, delegating interactive logic to dedicated client components (e.g., ExploreClient.tsx, EventDetailClient.tsx).
- **API Engine:** RESTful route handlers in app/api/**/route.ts handle client calls using standard HTTP verbs (GET, POST, PUT, PATCH).

3 Important Pages and Routes

ROUTE	ACCESS	DESCRIPTION	KEY SOURCE FILE
/	PUBLIC	Landing page with Hero, Delhi NCR zones, event vibe categories, and featured party previews	app/page.tsx
/explore	PUBLIC	Browse approved events with interactive multi-filter bar (city, vibe, price, date, query)	app/explore/ExploreClient.tsx
/events/[id]	PUBLIC	Event details screen showing vibe tags, capacity bar, host card, and Book Your Spot CTA	app/events/[id]/EventDetailClient.tsx
/login	PUBLIC	Email/password sign-in with automatic routing to /onboarding or /explore	app/login/page.tsx
/signup	PUBLIC	Account registration passing name metadata to Supabase DB triggers	app/signup/page.tsx
/onboarding	PROTECTED	4-step wizard collecting name, age, city/area, favorite event vibes, and contact	app/onboarding/page.tsx
/profile	PROTECTED	Account management hub to view role badge and update location, bio, and vibes	app/profile/page.tsx
/bookings	PROTECTED	My Bookings list showing confirmed/cancelled passes with status pills	app/bookings/page.tsx
/bookings/[id]	PROTECTED	Digital ticket pass displaying time, venue area, snapshot amount, and booking UUID	app/bookings/[id]/page.tsx
/host	PROTECTED	Host submission portal (HostForm) requiring	app/host/page.tsx

ROUTE	ACCESS	DESCRIPTION	KEY SOURCE FILE
		authentication before submission	
/host/dashboard	HOST/ADMIN	Host management portal showing hosted events, capacity bars, and booking counts	app/host/dashboard/page.tsx
/host/dashboard/[id]/guests	HOST/ADMIN	Attendee roster for a specific event with privacy controls (hides phone/email from hosts)	app/host/dashboard/[eventId]/guests/page.tsx
/admin	ADMIN ONLY	Admin portal approval queue with one-click Approve/Reject actions	app/admin/page.tsx

4 Reusable Components and Design System

- **Atomic Design System** (components/ui/): Button , Input , Select , Textarea , Modal , SectionHeading . Standardized with accessible HTML attributes and unified focus states.
- **Layout Shell** (components/layout/): NavBar (handles dynamic role badges and authenticated state), MobileNav (slide-out menu for smartphones), and Footer .
- **Context Provider** (components/providers/AuthProvider.tsx): Subscribes to onAuthStateChange , caches session tokens, and validates profile completion status via isProfileComplete() .
- **Discovery & Host Modules**: EventCard , EventGrid , EventGridSkeleton , FilterBar , and HostForm .

5 Supabase Client Architecture

CLIENT IDENTIFIER	LOCATION	SECURITY ROLE	PRIMARY USE CASE
createBrowserClient()	lib/supabase/client.ts	Anon Key (Browser)	Browser session storage, password login/signup, onAuthStateChange listener
getAuthUser(request)	lib/supabase/server.ts	Server Auth Validator	Extracts JWT Bearer token from header/cookie; cryptographically resolves user ID
supabase	lib/server/supabase.ts	Anon Key (Server)	Public server-side reads respecting standard Row Level Security
supabaseAdmin	lib/server/supabase.ts	Service Role Key	Bypasses RLS for atomic booking RPC, admin reviews, and profile backfills

6 Authentication and Onboarding Flow

1. **Account Registration:** User signs up via `supabase.auth.signUp()`. The PostgreSQL trigger `on_auth_user_created` fires immediately, creating a row in `public.profiles`.
2. **Role Enforcement:** The database trigger explicitly forces `role = 'user'`. Clients cannot specify or escalate roles during signup.
3. **Profile Completeness Check** (`lib/utils/profile.ts`): Evaluates whether required fields are set: `name` (`>= 2` chars), `ageRange`, `city`, `area`, and at least 1 `interest`. Incomplete profiles are routed to the 4-step wizard at `/onboarding`.
4. **Zero-Trust Token Extraction:** API routes never trust client-submitted user IDs; they derive the user identity directly from the verified Supabase JWT Bearer token via `getAuthUser(request)`.

7 Database Tables and Schemas

TABLE	PRIMARY KEY	KEY FIELDS & CONSTRAINTS	RLS ENABLED
<code>public.profiles</code>	<code>id</code> UUID (FK <code>auth.users</code>)	<code>name</code> , <code>email</code> , <code>phone</code> , <code>city</code> , <code>area</code> , <code>age_range</code> , <code>gender</code> , <code>avatar_url</code> , <code>instagram</code> , <code>interests</code> (TEXT[]), <code>role</code> ('user' 'host' 'admin')	YES
<code>public.events</code>	<code>id</code> UUID	<code>host_id</code> (FK <code>profiles</code>), <code>title</code> , <code>event_type</code> , <code>city</code> , <code>area</code> (approximate only), <code>date</code> , <code>start_time</code> , <code>price</code> , <code>capacity</code> , <code>spots_left</code> , <code>status</code> ('pending' 'approved' 'rejected')	YES
<code>public.host_submissions</code>	<code>id</code> UUID	<code>user_id</code> (FK <code>profiles</code>), <code>host_id</code> (FK <code>profiles</code>), <code>name</code> , <code>phone</code> , <code>event_title</code> , <code>city</code> , <code>area</code> , <code>date</code> , <code>capacity</code> , <code>price</code> , <code>vibe</code> , <code>description</code> , <code>status</code>	YES
<code>public.bookings</code>	<code>id</code> UUID	<code>event_id</code> (FK <code>events</code>), <code>user_id</code> (FK <code>profiles</code>), <code>amount</code> , <code>status</code> ('confirmed' 'cancelled' 'refunded'), <code>payment_status</code> ('test_paid') <i>Partial UNIQUE(event_id, user_id) WHERE status='confirmed'</i>	YES
<code>public.event_interests</code>	<code>id</code> UUID	<code>event_id</code> (FK <code>events</code>), <code>user_id</code> (FK <code>profiles</code>), <code>name</code> , <code>phone</code> , <code>age_range</code> , <code>city</code> , <code>area</code> , <code>instagram</code> , <code>reason</code> <i>UNIQUE(event_id, user_id)</i>	YES

8 User Roles and Role Handling

- **Roles:** `user` (standard attendees/submitters), `host` (party creators), `admin` (platform curators).
- **Environment Override:** Configured via `ADMIN_EMAILS` in `.env.local`. Evaluated by `isConfiguredAdminEmail()` in `lib/server/admin.ts`.
- **Automatic Elevation & Sync:** When a configured admin email logs in, `profileRepository.getById()` returns `role: 'admin'` and asynchronously synchronizes `role = 'admin'` into the database table.
- **Tamper Resistance:** RLS policy `profiles_update_own` strictly prevents users from mutating their own `role` column.

9 Event Creation Lifecycle

1. An authenticated user navigates to `/host` and fills out `HostForm`.
2. Form submits to `POST /api/host-submissions`. The server validates the payload and binds the user ID via `user.id` extracted from the JWT token.
3. A record is inserted into `public.host_submissions` with `status = 'pending'`.
4. The host is informed that the event is pending review (proposals are never published directly).

10 Event Approval and Publication Lifecycle

1. Admin visits `/admin`. The endpoint `GET /api/admin/events` returns all pending submissions.
2. When Admin clicks **Approve**:
 - Sends `PATCH /api/admin/events` with status `"approved"`.
 - Calls `eventRepository.createFromSubmission()`, creating a new record in `public.events` with `status = 'approved'` and retaining `host_id`.
 - Sets submission status to `'approved'`. The event instantly appears on `/explore`.
3. When Admin clicks **Reject**: Sets submission status to `'rejected'`. No event is created.

11 Host Dashboard Capabilities

- Located at `/host/dashboard`; accessible to users with `role === 'host' or 'admin'`.
- Queries events where `host_id = auth.uid()` and joins confirmed booking counts.
- Renders real-time capacity progress indicators (amber when `<= 5` spots remain; red when sold out).
- Provides guest list at `/host/dashboard/[eventId]/guests` with verified ownership check and privacy mask.

12 Admin Dashboard Capabilities

- Located at `/admin`; protected by server-side verification in `app/api/admin/events/route.ts`.
- Provides a real-time review queue of all submitted parties including cover pictures, host contact, venue area, pricing, and vibe descriptors.
- Includes duplicate approval safeguards and live status refreshes.

13 Row Level Security (RLS) & Atomic Engine

- **Atomic Booking Guarantee**: Implemented via `reserve_booking()` stored procedure (`supabase/migrations/20260828000005_bookings.sql`). Uses `SELECT ... FOR UPDATE` on the event row, serializing concurrent requests and eliminating race conditions or overbooking.
- **Public Data Protection**: Anonymous visitors can only read events where `status = 'approved'`. Host submissions and unapproved events remain invisible.
- **Data Isolation**: Users can only select and cancel their own bookings. Hosts can only view attendees for events they own.

14 Admin Access Diagnostic and Resolution

Root Cause Analysis: The user email `mohit616gigabyte@gmail.com` is correctly configured in `.env.local`. However, Next.js evaluates `process.env` variables when the development server boots. If the dev server was started before `ADMIN_EMAILS` was added or updated, Node.js retains the older cached environment state.

Action Steps (No Code Edits Required):

1. In your terminal running the Next.js dev server, press `Ctrl + C` and run `npm run dev`.
2. In your web browser, log out and log back in with `mohit616gigabyte@gmail.com` (or refresh the page).
3. The **Admin** link will appear in the top navbar, and navigating to `/admin` will grant full access to the approval queue.